Introduction to Monotonic Stack

Definition of Monotonic Stack

A Monotonic Stack is a special kind of stack, a common data structure in computer science, which maintains its elements in a specific order. Unlike a traditional stack, where elements are placed on top of one another based on when they arrive, a Monotonic Stack ensures that the elements inside the stack remain in an increasing or decreasing order. This is achieved by enforcing specific push and pop rules, depending on whether we want an increasing or decreasing monotonic stack.

Relevance in Coding Interviews

Monotonic Stacks are powerful tools in coding interviews due to their unique capabilities. They are particularly effective when it comes to problems requiring us to find the next smaller or larger element in a sequence, often referred to as Next Greater Element (NGE) or Next Smaller Element (NSE) problems.

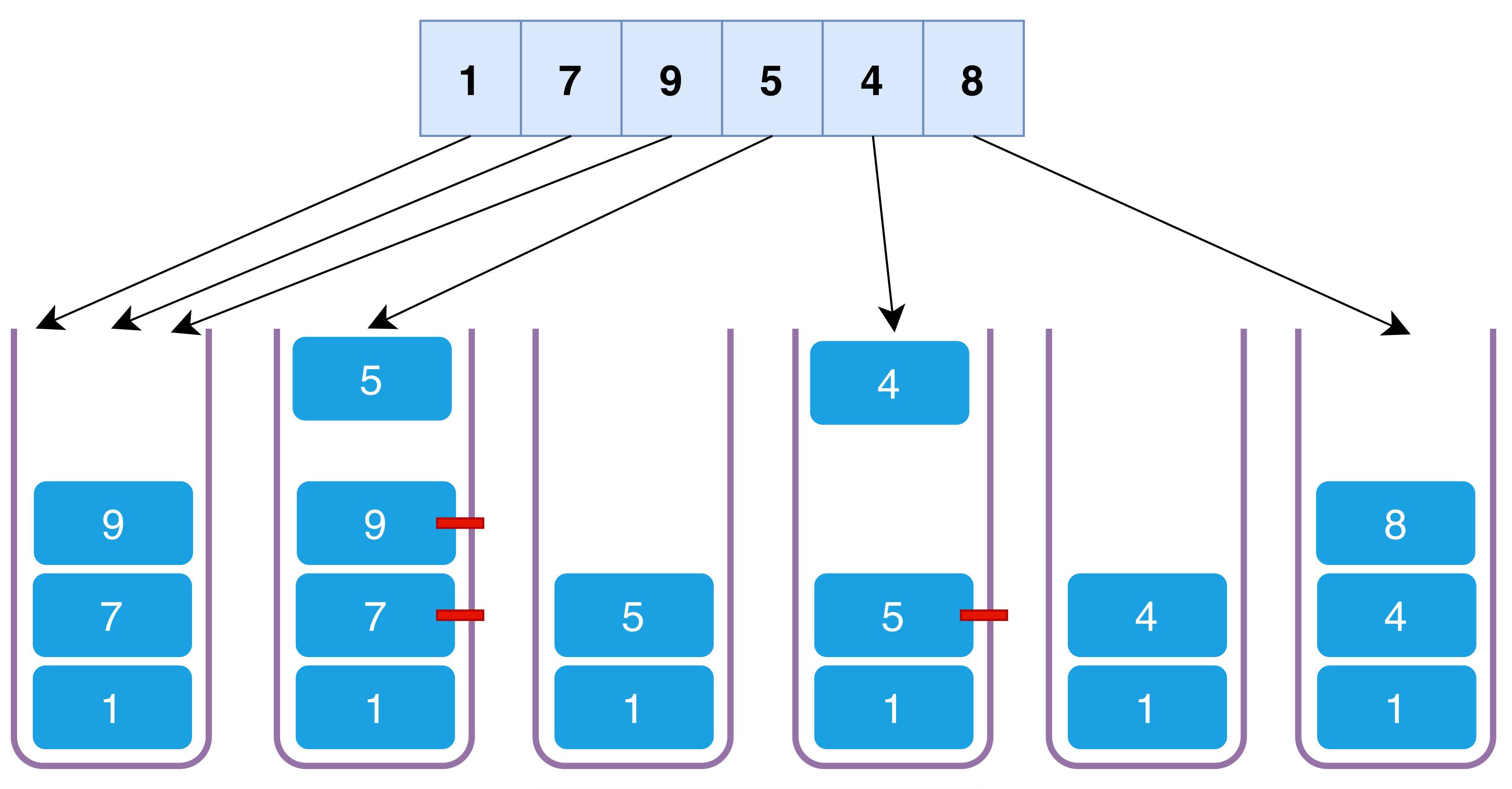
In the context of an interview, understanding and implementing a Monotonic Stack can not only help you solve the problem at hand but also demonstrate a strong grasp of data structures and algorithms to your interviewer. It shows that you can identify and apply the right tool for the task, a crucial ability for any software engineer.

Types of Monotonic Stacks

Monotonic Stacks can be broadly classified into two types:

1. **Monotonically Increasing Stack**

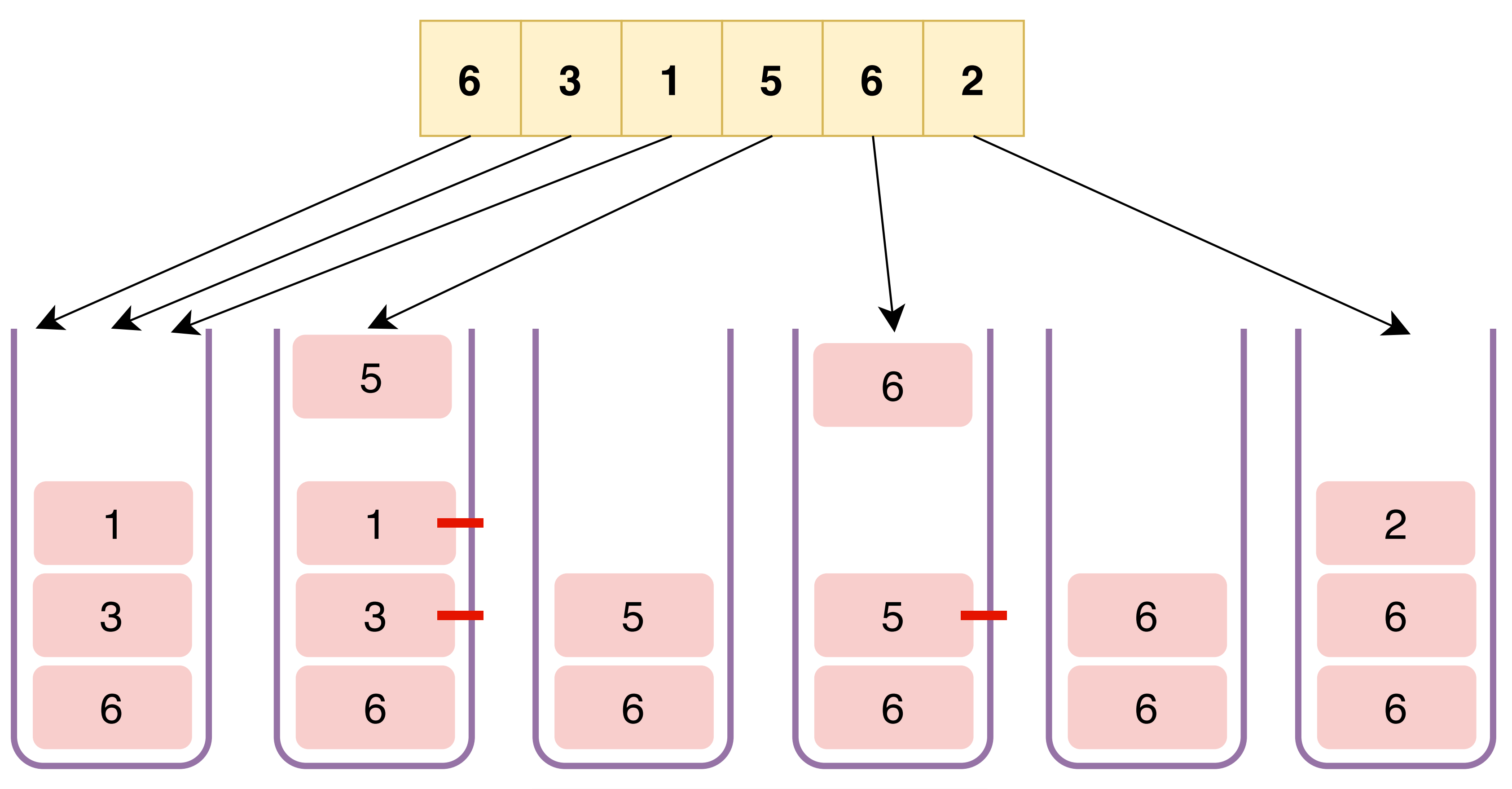
A Monotonically Increasing Stack is a stack where elements are arranged in an ascending order from the bottom to the top. Here, every new element that's pushed onto the stack is greater than or equal to the element below it. If a new element is smaller, we pop the elements from the top of the stack until we find an element smaller than or equal to the new element, or the stack is empty. This way, the stack always maintains an increasing order.



Monotonically Increasing Stack

2. **Monotonically Decreasing Stack**

Conversely, a Monotonically Decreasing Stack is a stack where elements are arranged in a descending order from the bottom to the top. When a new element arrives, if it's larger than the element on the top, we keep popping the elements from the stack until we find an element that's larger than or equal to the new element, or the stack is empty. This ensures that the stack always maintains a decreasing order.



Monotonically Decreasing Stack

Identifying Problems Suitable for Monotonic Stack

Recognizing when to use a Monotonic Stack is a vital skill. Here are some key aspects to consider:

Problem Characteristics

Monotonic Stacks are typically useful when dealing with problems that involve analyzing sequences or arrays, especially when you need to find the next or previous larger or smaller element for each element in the array. If you encounter a problem where the solution seems to require some sort of sequential step-by-step comparison, it's likely a good candidate for using a Monotonic Stack.

Example Scenarios

One classic sign that a Monotonic Stack might be helpful is when the problem description mentions finding the "next greater element" or the "next smaller element" in an array. Problems that involve finding maximum areas, such as in histograms, can also be solved effectively using Monotonic Stacks. Remember, the key is to identify patterns where a sequential step-by-step comparison is necessary.

Constructing Monotonic Stacks

Understanding how to build Monotonic Stacks is key. We'll break down this process for each type:

Code Template for Monotonically Increasing Stack

Here's a general structure of a Monotonically Increasing Stack in pseudo-code:

```
create an empty stack
for each element in the array:
    while stack is not empty AND top of stack is more than the current element:
        pop the stack
    push the current element to stack
```

This logic guarantees that for each element, all larger elements preceding it get popped, leaving the next smaller element (if it exists) at the top of the stack.

Code Template for Monotonically Decreasing Stack

Similarly, here's a template for a Monotonically Decreasing Stack:

```
create an empty stack
for each element in the array:
    while stack is not empty AND top of stack is less than the current element:
        pop the stack
    push the current element to stack
```

This structure ensures that for each element, all smaller elements preceding it get popped, leaving the next larger element (if it exists) at the top of the stack.

Explanation of the Code Templates

Both these templates work in a similar fashion. They loop through each element in the array, and for each one, they pop out the elements from the stack that are smaller (for increasing stack) or larger (for decreasing stack) than the current element. This ensures the stack stays monotonically increasing or decreasing.

Understanding Time Complexity

Grasping the time complexity of Monotonic Stacks is critical for efficiency. Let's break it down for both types:

Time Complexity for Monotonically Increasing Stack

In a Monotonically Increasing Stack, each element from the input array is pushed and popped from the stack exactly once. Therefore, even though there is a loop inside a loop, no element is processed more than twice. Hence, the time complexity of building a Monotonically Increasing Stack is $O(N)$, where N is the number of elements in the array.

Time Complexity for Monotonically Decreasing Stack

The situation is similar for a Monotonically Decreasing Stack. Each element is processed only twice, once for the push operation and once for the pop operation. As a result, the time complexity remains linear - $O(N)$, with N being the size of the array.

To summarize, although the construction of Monotonic Stacks might look complex at first glance, they are impressively efficient. Each element in the input array is handled only twice (one push and one pop), making the overall time complexity linear.

Next, we'll explore some practice problems to help you get more comfortable with Monotonic Stacks.

← Previous
Next →
Solution: Simplify Path
Next Greater Element (Easy)
☐ Mark as Completed